


Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội

Khoa Công nghệ Thông tin

Giao Thức SSL/TLS: Kiến Trúc, Tiến Hóa và Cơ Chế Bảo Mật Hiện Đại

Học phần: Mật mã và An toàn Thông tin

Sinh viên thực hiện:

Lê Minh Sơn
Mai Tiến Dũng
Nguyễn Nho Dương

Giảng viên:

TS. Lê Thị Hợi

Ngày:

13 tháng 11 năm 2026

Cấu Trúc Nội Dung Thuyết Trình

Mục Tiêu Trọng Tâm

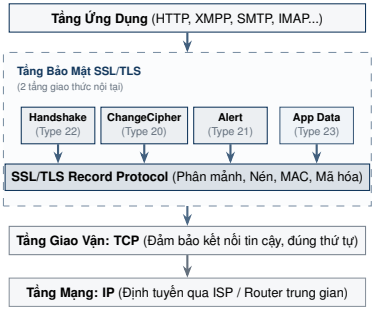
- **Định vị kiến trúc** SSL/TLS và lộ trình đào thải chuẩn cũ.
- **Nắm vững tầng Record**, quản lý trạng thái và xác thực PKI X.509 v3.
- **Đối sánh bắt tay** 0/1/2-RTT, cơ chế PFS và nâng cấp TLS 1.3.
- **Hiểu mở rộng DTLS** trên UDP và bảo vệ riêng tư SNI với ECH.
- **Đánh giá rủi ro lượng tử** và định hướng chuyển dịch PQ-TLS.

Khái Niệm & Định Vị SSL/TLS trong Mô Hình TCP/IP

Khái Niệm, Định Nghĩa & Vị Trí Kiến Trúc

- **SSL (*Secure Sockets Layer*)**: Netscape phát triển (1994), bảo mật Web.
- **TLS (*Transport Layer Security*)**: Chuẩn IETF kế thừa, thay thế SSL.
- **Mục tiêu**: Kênh end-to-end, bảo đảm bí mật, toàn vẹn và xác thực.
- **Vị trí TCP/IP**: Nằm giữa TCP và Ứng dụng; chạy ở *user-space*.
- **Đánh đổi vs. IPsec**: IPsec bảo vệ toàn mạng; SSL/TLS cô lập ở 2 đầu mút.

Vị Trí Kiến Trúc Hai Tầng của SSL/TLS



Bốn Dịch Vụ Bảo Mật Cốt Lõi của SSL/TLS

Trụ cột trên TCP: Kết hợp **bất đối xứng** (xác thực, trao đổi khóa) và **đối xứng** (mã hóa, toàn vẹn, chống phát lại).

1. Tính Bí Mật



Confidentiality

- **Mục tiêu**: Chống nghe lén trên đường truyền.
- **Khóa phiên**: Mã hóa đối xứng tạo lúc bắt tay.
- **Thuật toán**: `AES`, `ChaCha20`; *padding* che độ dài.

2. Tính Toàn Vẹn



Message Integrity

- **Mục tiêu**: Phát hiện sửa đổi hoặc cắt gói tin.
- **MAC**: `HMAC` (`SHA-256`, `SHA-384`).
- **Chuẩn mới**: Tích hợp nguyên tử **AEAD** (`GCM`).

3. Xác Thực



Authentication

- **Mục tiêu**: Chống mạo danh máy chủ, đầu cuối.
- **Chiều**: Server bắt buộc; Client tùy chọn (`mTLS`).
- **Cơ chế**: `RSA`, `ECDSA` + X.509 hoặc `PSK`.

4. Chống Phát Lại



Replay Protection

- **Mục tiêu**: Ngăn phát lại gói tin hợp lệ.
- **Bắt tay**: Cặp nonces ngẫu nhiên độc lập.
- **Truyền tải**: Đánh số `sequence_number` 64-bit; `0-RTT early_data` vẫn có số thứ tự nhưng phát lại được giữa các kết nối.

Khởi Nguyên tại Netscape: Từ SSL 1.0 đến SSL 3.0

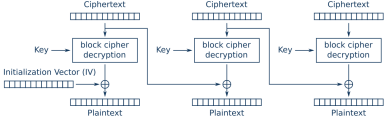
Khởi nguồn (1994): Netscape thiết kế SSL cho thương mại điện tử (Taher Elgamal). **SSL 1.0** chỉ dùng nội bộ do tồn tại lỗi an toàn nghiêm trọng.

SSL 2.0 (02/1995) – Cấm RFC 6176 (03/2011)

- ✗ **Chung khóa Enc & MAC**: Dùng 1 khóa cho cả 2 tác vụ.
- ✗ **MAC MD5 yếu**: Dễ bị tấn công mở rộng độ dài.
- ✗ **Bắt tay hở**: MITM dễ ép hạ cấp bộ mã (*downgrade*).
- ✗ **Thiếu** `close_notify`: Cắt cụt phiên bởi `FIN` giả; 1 IP/host.

SSL 3.0 (11/1996) – Khai Tử RFC 7568 (06/2015)

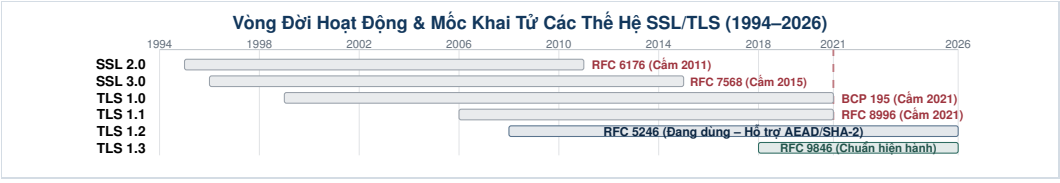
- ✓ **Kiến trúc nền**: Tách Record/Handshake, tách khóa, thêm `Finished` & `close_notify`.
- ✗ **Lỗ hổng POODLE (2014)**: Lỗi đệm khối CBC → giải mã từng byte bản rõ.
- ✗ **RC4 yếu & thiếu Extensions**: RC4 lệch thống kê; không có trường mở rộng để vá CBC.



Tại sao CBC là tử huyệt của SSL 3.0?

Giải mã chuỗi khối $P_i = D_K(C_i) \oplus C_{i-1}$ cho phép kẻ tấn công sửa byte đệm của C_{i-1} để dò từng byte bản rõ (**POODLE**). Do không có *Extensions* vá lỗi, RFC 7568 khai tử hoàn toàn SSL 3.0.

Chuẩn Hóa IETF (TLS 1.0 – 1.3) & Khai Tử Thế Hệ Cũ



4 Thế Hệ Chuẩn Hóa TLS tại IETF

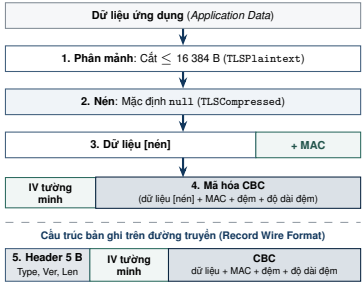
- **TLS 1.0 (1999, RFC 2246)**: Chuẩn hóa HMAC, PRF (`MD5+SHA-1`); yếu do IV ngầm định CBC (BEAST).
- **TLS 1.1 (2006, RFC 4346)**: Explicit IV cho bản ghi CBC; ẩn lỗi đệm chống tấn công định thời.
- **TLS 1.2 (2008, RFC 5246)**: PRF SHA-2, AEAD (`AES-GCM`); RFC 10015 (2026) loại RSA tĩnh/DHE.
- **TLS 1.3 (2018/2026, RFC 8446 → 9846)**: Bắt buộc AEAD, HKDF; PFS phụ thuộc khóa tạm; bỏ CBC, SHA-1, RSA tĩnh.

3 Lý Do Cấm TLS 1.0 & 1.1 (BCP 195)

1. **Thiếu AEAD**: Buộc duy trì bộ mã khối CBC cũ như `3DES-EDE-CBC-SHA`.
2. **Phụ thuộc MD5 + SHA-1**: Hàm PRF và chữ ký bắt tay gắn cứng hai hàm băm đã bị phá (va chạm SHA-1 $\approx 2^{63}$).
3. **Phình đại mã nguồn**: Tăng bề mặt tấn công; PCI DSS yêu cầu loại bỏ từ 2018.

Record Protocol: 5 Bước Đóng Gói (TLS 1.2 CBC)

Sơ Đồ Đóng Gói TLS 1.2 CBC (RFC 5246, mục 6.2.3.2)



Header 5 B | IV tường minh | CBC(dữ liệu + MAC + đệm + độ dài đệm).

5 Công Đoạn Tuần Tự (TLS 1.2 Dùng CBC)

1. **Phân mảnh**: Cắt dữ liệu thành từng mảnh $\leq 2^{14}$ B (16 384 B) dạng `TLSPlaintext`.
2. **Nén**: Mặc định `null`, loại rò rỉ độ dài (chống **CRIME**).
3. **Gắn MAC**: Tính `HMAC` bằng `write_mac_secret` kèm `sequence_number` 64-bit ngầm định.
4. **IV & Mã hóa CBC**: Gắn **IV tường minh** (dạng rõ) và mã hóa `CBC(dữ liệu [nén] + MAC + đệm + độ dài đệm)`; tổng $\leq$ 18 432 B.
5. **Record Header (5B)**: Content Type (1B: `CCS`, `Alert`, `Handshake`, `App`), Version (2B), Length (2B).

AEAD (TLS 1.2 & TLS 1.3): TLS 1.2 cũng hỗ trợ AEAD (không MAC rời/CBC). Ở TLS 1.3 (bắt buộc AEAD, bỏ nén), bản ghi đã mã hóa (`TLSCiphertext`) đặt `Type = 23` để che loại bản tin thật [RFC 9846, mục 5].

Giao Thức Quản Lý: Change Cipher Spec & Alert

1. Change Cipher Spec Protocol (Content Type = 20)

Type = 20	Ver (2B)	Len = 1	Payload = 1

Giao thức riêng trên Record (không thuộc Handshake).

- **Chuyển trạng thái**: Báo hiệu đầu cuối sao chép `Pending State` đè sang `Current State`.
- **Kích hoạt mật mã**: Áp dụng khóa và thuật toán mới ngay từ bản ghi kế tiếp (`Finished`); đặt lại `sequence_number` về 0.
- **Trong TLS 1.3**: Bỏ vai trò chuyển khóa; chỉ gửi bản tin giả lập nhằm tương thích thiết bị mạng trung gian (*middlebox*).

2. Alert Protocol (Content Type = 21)

Cấu Trúc Bản Tin 2 Byte

Byte 1: Severity	Byte 2: Alert Code

Byte 1: `1 = Warning`, `2 = Fatal`.
Bảo vệ: Được nén và mã hóa theo trạng thái phiên hiện hành.

Mức Fatal (Level 2 – Hủy Phiên)

- Chấm dứt kết nối ngay, xóa khóa phiên và cấm nối lại phiên đó.
- `bad_record_mac`: Lỗi MAC/đệm CBC.
- `handshake_failure`: Lệch tham số mã.
- `unexpected_message`: Sai thứ tự pha.

Mức Warning (Level 1 – Cảnh Báo)

- `close_notify`: Báo đóng phiên chuẩn 2 chiều, chống cắt cụt bằng cờ `TCP FIN` giả mạo.
- **Chứng thư X.509**: `bad_certificate`, `certificate_expired`, `certificate_revoked` (TLS 1.2; TLS 1.3 luôn Fatal).

Quản Lý Trạng Thái: Session vs. Connection State

TLS 1.2: Chuyển Trạng Thái Nguyên Tử (Pending State → Current State qua CCS)



Tiêu chí đối chiếu	Trạng Thái Phiên (Session State)	Trạng Thái Kết Nối (Connection State)
Bản chất & Vòng đời	Liên kết logic **lâu dài** tạo qua Handshake; dùng chung cho nhiều kết nối.	Liên kết truyền tải **tạm thời** (*transient*); gắn với 1 phiên tại mỗi thời điểm.
Mục đích vận hành	Cho phép nối lại phiên (*Session Resumption*) nhanh, giảm độ trễ bắt tay.	Trực tiếp thực thi mã hóa, giải mã và kiểm tra MAC độc lập cho 2 chiều Đọc/Ghi.
Nhóm tham số định danh & thuật toán	**1.** `Session Identifier` (Mã phiên) **2.** `Peer Certificate` (Chứng thư X.509) **3.** `Compression` & **4.** `Cipher Spec`	**1.** `Server Random` & **2.** `Client Random` (Nonces) **3.** `Server/Client Write MAC Secret` **4.** `Server/Client Write Key` (Khóa đối xứng)
Nhóm tham số bí mật & đồng bộ	**5.** `Master Secret` (48 byte dùng chung) **6.** Cờ `Is Resumable` (Cho phép nối phiên)	**5.** `Initialization Vectors (IV)` (Chế độ CBC/AEAD) **6.** `Sequence Number` 64-bit (Đặt về 0 khi CCS)

Hạ Tầng PKI & Cấu Trúc Chứng Thư Số X.509 v3

Đối Chiếu SAN & Chuỗi CA (Firefox / RFC 9525)



Đối chiếu URI `opensuse.org` ⟶ **SAN**; CA: `Let's Encrypt E5` (RFC 9525).

Chuỗi Tin Cậy PKI (Chain of Trust) Chống Tấn Công MITM



1. Nguyên Tắc, Định Dạng & Trường Chuẩn

- **Chống tráo khóa**: Gắn Public Key + danh tính (chứng thư công khai; Private Key lưu mật).
- **Định dạng tệp**: Nhị phân `DER`, Base64 `PEM`, chuỗi `PKCS#7`.
- **Trường chuẩn**: `Version v3`, `Serial`, `Issuer/Subject (DN)`, `Validity (Not Before/After)`, `Public Key`.

2. Toàn Vẹn Chữ Ký CA, Kiểm Tra SAN & Thu Hồi

- **Chữ ký số CA**: Bảo vệ toàn vẹn nội dung và chuỗi xác thực tới Root CA.
- **Kiểm tra SAN (RFC 9525)**: Trình duyệt xác minh chuỗi tin cậy, thời hạn và đối chiếu tên miền đang truy cập với trường **SAN** (thay `CN`).
- **Thu hồi & Bắt tay**: Tra cứu hủy hiệu lực qua `CRL`/`OCSP` (hoặc OCSP Stapling).

Bắt Tay TLS (0 / 1 / 2 RTT) & Phạm Vi Độ Trễ

Chế độ bắt tay	Khi nào xuất hiện / Sử dụng	Độ trễ	Đặc trưng mật mã & Điểm cần nhớ
TLS 1.2 Full	Kết nối mới đầy đủ (VD: dùng RSA tĩnh)	2 RTT	RSA tĩnh không có PFS; lộ khóa riêng RSA về sau làm lộ toàn bộ bản ghi cũ.
TLS 1.2 Resumption	Tiếp nối phiên rút gọn (*abbreviated handshake*)	1 RTT	Tái sử dụng `master_secret` qua Session ID/Ticket; 2 RTT chỉ là luồng đầy đủ điển hình.
TLS 1.3 (1-RTT)	Bắt tay mới với khóa tạm `(EC)DHE` (chuẩn hiện đại)	1 RTT	Chuẩn **RFC 9846**; luồng dùng `(EC)DHE` có PFS (2 RTT nếu có `HRR`; `PSK-only` thiếu PFS).
TLS 1.3 (0-RTT)	Tiếp nối phiên bằng PSK / ticket khi được bật	0 RTT	Gửi `early_data` ngay lượt đầu; `early_data` thiếu PFS và phát lại được ngay cả khi dùng `PSK + ECDHE`.

1. Định Nghĩa & Phạm Vi Đo

- **RTT (*Round-Trip Time*)**: Thời gian 1 lượt đi–về mạng (không phải số bản tin).
- **Phạm vi chuẩn**: Đo trên TCP đã kết nối; không cộng DNS, bắt tay TCP (+1 RTT), xử lý hay QUIC.

2. Giả Định Khi Đếm RTT

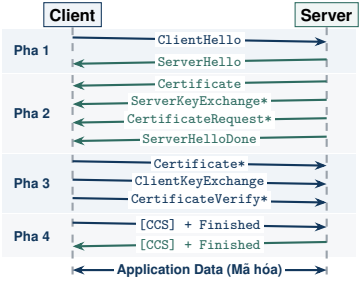
- **Không giả định *False Start***: Tính theo quy trình chuẩn RFC của TLS 1.2 và TLS 1.3.
- **Mốc gửi dữ liệu 1-RTT**: Tính lúc client bắt đầu gửi dữ liệu ứng dụng (server nhận `Finished` muộn hơn).

3. Ví Dụ Mạng RTT = 50 ms



Bắt Tay Đầy Đủ TLS 1.2 (2-RTT): Cấu Trúc 4 Pha

Luồng Bản Tin Bắt Tay (TLS 1.2 Full)



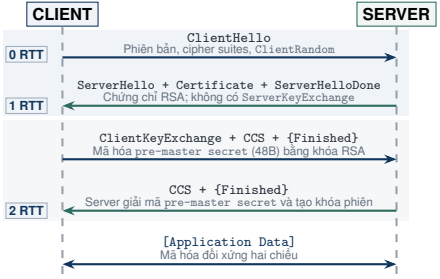
Pha 1–2: lượt 1 (1 RTT); Pha 3–4: lượt 2 (2 RTT). Dấu ∗: tùy chọn (mTLS / KEX).

Ý Chính Của 4 Pha Đàm Phán (2 RTT)

1. **Pha 1 – Đàm phán năng lực**: Gửi `ClientHello` & `ServerHello` chốt phiên bản, nonces ngẫu nhiên và chọn bộ mã.
2. **Pha 2 – Xác thực Server & Khóa tạm**: Server gửi chứng thư X.509, tham số khóa tạm (`ECDHE*`) ký số và chốt bằng `ServerHelloDone`.
3. **Pha 3 – Trao đổi khóa & Client**: Client thẩm tra chứng thư, gửi khóa (`ClientKeyExchange`) và xác thực mTLS nếu cần.
4. **Pha 4 – Kích hoạt & Hoàn tất**: Hai bên gửi `ChangeCipherSpec` bật mã hóa, rồi gửi `Finished` xác thực toàn bộ lịch sử bắt tay.

TLS 1.2 (2-RTT): RSA Key Transport vs. (EC)DHE

Hình 1: TLS 1.2 · RSA Key Transport (2 RTT)



CCS = ChangeCipherSpec; {} = mã hóa bắt tay; [] = khóa ứng dụng.
[RFC 5246, mục 7.3]

RSA Thực Sự Mã Hóa Dữ Liệu Nào?

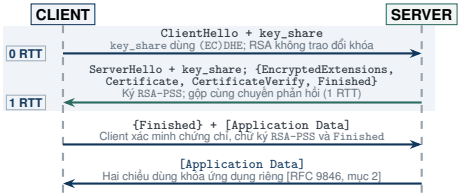
- **Bí mật 48B**: Client tạo `pre-master secret` (2B bản + 46B ngẫu nhiên), mã hóa bằng `RSAES-PKCS1-v1_5` (không phải `OAEP`).
- **Dẫn xuất & EMS**: `pre-master` → `master secret` → `key block`; bật EMS (RFC 7627) gắn PRF với hash transcript.
- **Suite** `TLS_RSA_WITH_AES_128_GCM_SHA256`: `AES-GCM` mã hóa bản ghi, `SHA-256` cho PRF; xác nhận qua `Finished`.

Khác Biệt Trên Luồng Bản Tin Trong TLS 1.2

- **RSA key transport (✗ Không PFS)**: Bỏ qua `ServerKeyExchange`; server dùng khóa riêng dài hạn giải mã `pre-master` (rủi ro Bleichenbacher).
- **Ephemeral** `(EC)DHE` **(✓ Có PFS)**: Server gửi `ServerKeyExchange` chứa khóa DH tạm ký số; **RFC 10015 (2026)** loại RSA tĩnh & DHE hữu hạn.
- **Abbreviated Resumption (1 RTT)**: Bỏ Pha 2–3 qua Session ID/Ticket, tái dùng `master_secret` (không chạy lại DH mới).

TLS 1.3 (1-RTT): Chứng Chỉ RSA & (EC)DHE

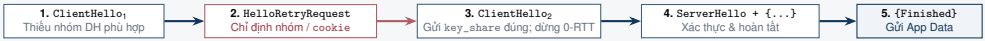
Hình 2: TLS 1.3 · Chứng Chỉ RSA · Bắt Tay 1-RTT



Chứng Chỉ RSA ≠ RSA Key Transport

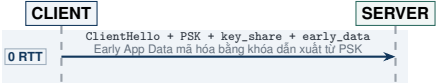
- **RFC 9846 (07/2026, thay RFC 8446)**: Loại bỏ hoàn toàn RSA key transport; không thể thay `key_share` bằng `pre-master secret` mã hóa RSA.
- **Tách xác thực & trao đổi khóa**: Server dùng khóa riêng RSA; `CertificateVerify` (RFC 9846, mục 4.5.2) dùng `RSA-PSS` ký dữ liệu chứa hash transcript đến hết `Certificate`.
- **PFS trong luồng** `(EC)DHE`: Bí mật phiên có đóng góp từ khóa tạm `(EC)DHE` (cũng như `PSK + ECDHE`); tiếp nối `PSK-only` (`psk_ke`, mục 4.3.9) thiếu khóa tạm.

Khi Nào TLS 1.3 Cần 2 RTT? Luồng `HelloRetryRequest` (RFC 9846, mục 4.2.4) — Khi chưa khớp nhóm `key_share` hoặc cần `cookie`



TLS 1.3 (0-RTT): Gửi Sớm & Đánh Đổi An Toàn

Hình 3: TLS 1.3 · 0-RTT · Tiếp Nối PSK + (EC)DHE

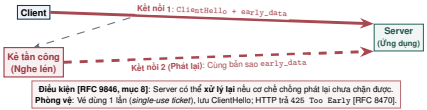


Xác thực bằng PSK (không cần chứng chỉ mới). `PSK-only` (`psk_ke`) bỏ `key_share` nên thiếu PFS [RFC 9846, mục 2.3 & 4.3.9].

Cơ Chế PSK & Phân Biệt Hai Lớp Khóa

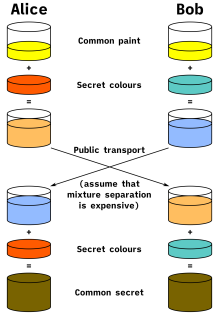
- **Dẫn xuất PSK**: Server gửi `NewSessionTicket`; client tạo PSK từ bí mật phiên cũ + ticket (ticket là danh tính, không phải PSK gửi lộ).
- **Điều kiện** `early_data`: Chỉ có ticket chưa đủ; server còn phải cho phép và đặt giới hạn kích thước.
- **Hai lớp khóa**: `early_data` chỉ suy từ PSK cũ nên **thiếu PFS** (còn `PSK-only` thiếu đóng góp khóa tạm cho cả phiên). Dữ liệu 1-RTT (`PSK + ECDHE`) có PFS nhờ DH mới.

Sơ Đồ Rủi Ro Phát Lại (Replay Attack) Trong 0-RTT



Bảo Mật Chuyển Tiếp (PFS): Trực Giác DH

Minh Họa Trộn Màu Diffie-Hellman



Nguyên lý thiết lập bí mật chung trên kênh công khai (Wikimedia Commons, Public Domain).

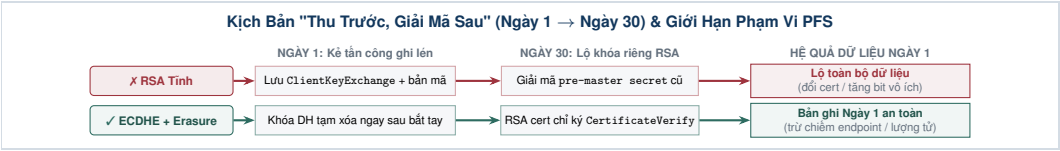
Cơ Chế Khóa Phù Du (Ephemeral DH / ECDHE)

- **Tham số công khai (*Common paint*)**: Hai bên chốt nhóm đường cong chuẩn (`X25519`, `P-256`) ngay trong bắt tay.
- **Khóa bí mật dùng 1 lần (*Secret colours*)**: Client & Server tự sinh số ngẫu nhiên (a, b) riêng cho từng phiên, tuyệt đối không dùng lại.
- **Tính một chiều**: Kẻ nghe lén thấy phần trộn công khai (g^a, g^b) nhưng bài toán Logarit rời rạc ngăn tách ngược (a, b) để tìm *Common secret*.

4 Bước Chứng Chỉ RSA Song Hành PFS

1. **Sinh cặp khóa DH tạm**: Client & Server tự tạo cặp khóa phù du (a, g^a) và (b, g^b) trên bộ nhớ RAM.
2. **Tính** `shared secret`: Trao đổi phần công khai tạm qua `key_share` / `ServerKeyExchange` để ra khóa phiên chung.
3. **Ký xác thực bằng chứng chỉ**: Server dùng khóa dài hạn (RSA/ECDSA) ký bản tin xác thực (`RSA-PSS` / `ECDSA`) chống MITM.
4. **Xóa khóa tạm (*Erasure*)**: Hủy ngay khóa riêng (a, b) sau khi sinh khóa phiên — khóa RSA bị lộ sau này cũng vô hiệu với phiên cũ.

Bảo Mật Chuyển Tiếp (PFS): Kịch Bản Thực Tế



Kịch bản	Thiết lập khóa phiên	Khi lộ khóa dài hạn	Trạng thái PFS
RSA tĩnh (TLS 1.2)	Mã hóa `pre-master` bằng RSA cert	Giải mã được `pre-master` cũ	✗ **Không PFS**: Lộ toàn bộ bản ghi cũ.
(EC)DHE (1.2 / 1.3)	Thỏa thuận `ECDHE/DHE` từng phiên	RSA cert chỉ ký xác thực	✓ **Có PFS**: Giữ bí mật phiên cũ (trừ lượng tử).
TLS 1.3 PSK-only	Tiếp nối bằng PSK, không DH mới	Phụ thuộc khóa ticket / PSK	✗ **Thiếu PFS** (cần `PSK + (EC)DHE`).
TLS 1.3 0-RTT	`early_data` gửi trước bước DH	Khóa 0-RTT suy từ PSK cũ	✗ `early_data` **thiếu PFS**.

Lưu Ý Giới Hạn Phạm Vi PFS: (1) PFS không ngăn được tấn công chiếm quyền điều khiển endpoint hoặc đánh cắp `Session Ticket Key` khi chưa xoay vòng khóa; (2) Khóa tạm `ECDHE` cổ điển vẫn có thể bị máy tính lượng tử bẻ ngược theo kịch bản *Store now, decrypt later* → cần kết hợp trao đổi khóa hậu lượng tử (`X25519MLKEM768`).

Nâng Cấp Bộ Mã Hóa (Cipher Suite): TLS 1.2 vs. 1.3



1. Tinh Giản Bộ Mã (Còn 5 Suites Chuẩn)

- Giảm từ hàng trăm bộ mã TLS 1.2 xuống **5 suite AEAD** (RFC 9846).
- Tách KEX & xác thực sang extension, giảm lỗi thương lượng.

2. Bắt Buộc AEAD & Loại Bỏ Thuật Toán Yếu

- Chỉ giữ `AES-GCM`, `ChaCha20-Poly1305`, `AES-CCM`.
- Loại `RC4`, `3DES`, `CBC`, MAC rời & nén (POODLE/CRIME).

3. Nâng Cấp Chữ Ký Số & Hàm Băm

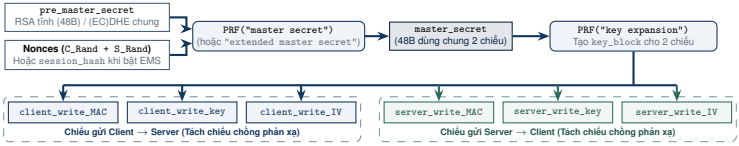
- Cấm `MD5`, `SHA-1` và ký `RSA PKCS#1 v1.5` ở bắt tay.
- Bắt buộc `RSA-PSS`, `ECDSA` (P-256/384) hoặc `Ed25519`.

4. Tách Trao Đổi Khóa Khỏi Chứng Chỉ

- Bỏ RSA key transport và DHE tùy biến yếu; dùng nhóm chuẩn (`X25519`, P-256).
- Đàm phán KEX độc lập loại khóa X.509; cấm chế độ ẩn danh (`ADH`).

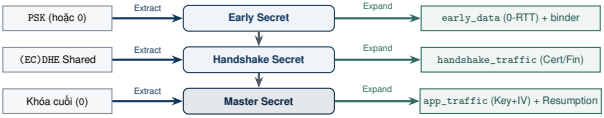
Dẫn Xuất Khóa: PRF (TLS 1.2) vs. HKDF (TLS 1.3)

TLS 1.2 (RFC 5246): Công Thức PRF Gốc (Trước EMS) & write_MAC Với Bộ Mã Cần MAC Rời



Hai chiều cùng sinh từ master_secret (RFC 7627, mục 4); bộ mã AEAD (RFC 5246, mục 6.3) bỏ write_MAC cho cả hai chiều.

TLS 1.3 (RFC 9846): Chuỗi Khóa Đa Tầng HKDF (*Extract & Expand*) Ràng Buộc Transcript



- **Tách khóa theo pha**: Mã hóa cả bản tin bắt tay (Cert/Verify) ngay sau ServerHello, gắn hash transcript.
- **Tinh gọn AEAD**: Chỉ sinh Key + IV cho 2 chiều; loại bỏ write_MAC rời.
- **Đổi khóa nhanh**: KeyUpdate tạo khóa mới từ app_traffic bằng HKDF-Expand (không chạy lại DH).

Đối Sánh TLS 1.2 vs. TLS 1.3 & Khuyến Nghị

Thành phần	TLS 1.2 (RFC 5246)	TLS 1.3 (RFC 9846 – Thay RFC 8446)
Cipher Suite	Gộp KEX + Auth + Enc + PRF vào 1 tên.	Chỉ chọn `AEAD + HKDF`; KEX & chữ ký đàm phán riêng.
Trao đổi khóa	Còn RSA tĩnh & DHE (bị loại ở **RFC 10015 (2026)**).	Bắt tay mới dùng khóa tạm như `(EC)DHE` hoặc **nhóm lai**; tiếp nối `PSK / PSK+DHE`.
Mã hóa Record	Còn `CBC`, MAC rời; AEAD là tùy chọn.	Chỉ giữ **AEAD** (`AES-GCM`, `ChaCha20-Poly1305`, `AES-CCM`).
Dẫn xuất khóa	`PRF` + 1 `master_secret` (48B) chung.	**HKDF** tách khóa handshake, app, exporter, PSK.
Bản tin bắt tay	Nhiều bản tin (chứng chỉ) truyền dạng rõ.	Mã hóa sau `ServerHello` (`SNI` vẫn cần ECH).
Bề mặt lỗi	Còn `compression`, `renegotiation`.	Loại bỏ hoàn toàn; thu gọn thương lượng & giảm RTT.

1. Ưu Tiên TLS 1.3



Bật mặc định **TLS 1.3 (RFC 9846)** để có 1-RTT, PFS và mã hóa handshake. Vô hiệu hóa SSL 2.0–TLS 1.1 (**BCP 195**).

2. Cấu Hình TLS 1.2



Nếu cần tương thích TLS 1.2, bắt buộc dùng `ECDHE` + **AEAD** và bật `EMS` (**RFC 9325**); loại RSA tĩnh/DHE (**RFC 10015**).

3. Luân Chuyển Khóa



Kích hoạt `KeyUpdate` định kỳ trên kết nối dài để giới hạn dữ liệu AEAD; lưu ý không thay thế khóa tạm mới (`(EC)DHE`/nhóm lai) nếu lộ `traffic secret`.

4. Kiểm Soát 0-RTT



Thẩm tra cert/tên miền/hạn. Nếu cần PFS cho *mọi* dữ liệu hoặc tránh rủi ro phát lại `0-RTT`, tắt 0-RTT và dùng **1-RTT** (`(EC)DHE`/nhóm lai).

Giao Thức DTLS: Động Lực & Cải Tiến Tầng Record

Lý Do Ra Đời & Rào Cản Trên UDP

- **Ứng dụng thời gian thực**: Hội nghị truyền hình, VoIP, game trực tuyến cần độ trễ thấp → dùng **UDP**, tái sử dụng hạ tầng TLS.
- **Record ngầm định**: TLS đếm thứ tự ngầm định; mất 1 record làm giải mã các record sau bị sai.
- **Thứ tự bắt tay**: Handshake TLS phải đúng thứ tự trên kênh tin cậy.
- **Rủi ro UDP**: Dễ bị DoS và khuếch đại (giả mạo IP nguồn).

Giải Pháp Tại Tầng Record (DTLS 1.2 → 1.3)

- **Số thứ tự tường minh**: Gắn trực tiếp `sequence_number` và `epoch` (thế hệ khóa) vào mỗi record để giải mã khi lệch thứ tự.
- **Chống phát lại (*Replay Detection*)**: Theo dõi bản ghi hợp lệ đã nhận bằng cửa sổ trượt bitmap (*bitmap window*).
- **Nâng cấp DTLS 1.3 (RFC 9147)**: Dùng *Unified Header* nén trường tiêu đề và thêm `Connection ID` giữ phiên khi đổi IP/NAT.

Tiêu Đề Record Tường Minh (DTLS 1.2) & Cơ Chế Cửa Sổ Trượt Bitmap (*Bitmap Window*)

1. Gắn Tường Minh vào DTLS Record Header

Type	Ver	Epoch	Seq Num	Len	Data
1B	2B	**2B**	**6B**	2B	Enc

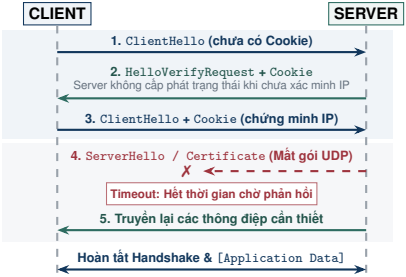
Thay thế bộ đếm ngầm định của TLS → xác định đúng thế hệ khóa & thứ tự bản ghi

2. Cơ Chế Chống Phát Lại Dựa Trên Cửa Sổ Bitmap



Bắt Tay DTLS: Cookie, Phân Mảnh & Đánh Giá

Chống DoS Bằng Cookie & Truyền Lại (DTLS 1.2)



DTLS 1.2 dùng `HelloVerifyRequest`; DTLS 1.3 hợp nhất thành `HelloRetryRequest` + cookie.

Phân Mảnh Thông Điệp Handshake ($\sim$ 1500 B)

message_seq	fragment_offset	fragment_length	Payload

- **Vượt kích thước UDP datagram**: Thông điệp bắt tay có thể lớn hơn 1 datagram $\rightarrow$ chia thành nhiều mảnh nhỏ.
- **3 thông tin ghép mảnh**: `message_seq` (thứ tự thông điệp), `fragment_offset` (vị trí bắt đầu) và `fragment_length` (độ dài mảnh).
- **Khôi phục**: Bên nhận ghép các mảnh lại thành thông điệp ban đầu.

Lợi Ích, Nhược Điểm & Tổng Kết DTLS

- ✓ **Lợi ích**: Giảm độ trễ thời gian thực; bảo mật datagram; tái dùng hạ tầng TLS; chống DoS & hỗ trợ mạng không dây.
- ✗ **Overhead trên từng gói**: Mỗi datagram phải gánh riêng Record Header (`Epoch`/`SeqNum`), Nonce và AEAD Tag.
- ✗ **Phân tách tin cậy**: Tầng DTLS xử lý timer truyền lại & ghép mảnh cho **handshake** (RFC 9147, mục 5); ứng dụng tự lo độ tin cậy cho dữ liệu ứng dụng nếu cần.
- **Tham chiếu**: **RFC 6347** (DTLS 1.2) & **RFC 9147** (DTLS 1.3).

Vấn Đề Lộ SNI & Giải Pháp ECH (RFC 9849)

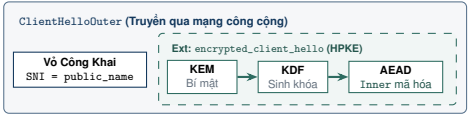
Vấn Đề: Rò Rỉ SNI Plaintext Trong TLS 1.3

- **Lỗ hổng còn lại**: Dù TLS 1.3 mã hóa phần lớn handshake (kể cả certificate), trường **SNI** (*Server Name Indication*) vẫn truyền dạng *plaintext*.
- **Lộ tên miền đích**: Thông điệp `ClientHello` ban đầu để lộ website người dùng muốn kết nối.
- **Theo dõi & Kiểm duyệt**: Kẻ nghe lén, nhà mạng (ISP) hoặc thiết bị trung gian theo dõi hành vi duyệt web và chặn theo SNI công khai.

Giải Pháp ECH: Tách Đôi ClientHello

- `ClientHelloInner` **(Thật – Được mã hóa)**: Chứa `SNI` thật, `ALPN` thật... cần bảo vệ khỏi các thực thể trung gian.
- `ClientHelloOuter` **(Vỏ ngoài – Công khai)**: Chứa giá trị vô hại (`SNI` là `public_name` của nhà cung cấp, VD tên miền chung của CDN) cùng extension `encrypted_client_hello` mang Inner đã mã hóa.

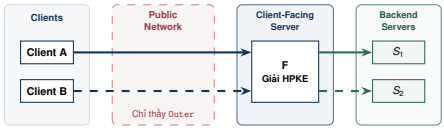
Cơ Chế HPKE (*Hybrid Public Key Encryption* – RFC 9180) Đóng Gói `ClientHelloInner`



Thành phần	Chức năng trong HPKE (RFC 9180)
KEM	Thiết lập bí mật dùng để mã hóa
KDF	Sinh khóa mã hóa từ bí mật
AEAD	Mã hóa và xác thực dữ liệu

Quy Trình 5 Bước ECH & Thách Thức Bảo Mật

Mô Hình Truyền Tải ECH Qua Client-Facing Server



Thách Thức & Cơ Chế Bảo Vệ (RFC 9849)

1. **Rò rỉ DNS rõ & IP đích**: Truy vấn DNS không mã hóa (thiếu DoH/DoT) hoặc IP đích vẫn lộ danh tính server.
2. **Chèn tham số** `ClientHello`: RFC 9849 (mục 10.12.3) ràng buộc `ClientHelloOuter` qua `AAD` để chặn sửa đổi.
3. **Theo dõi client (*tracking*)**: Server độc hại có thể phát `ECHConfig` riêng cho từng client.

Quy Trình Hoạt Động Chi Tiết (5 Bước)

1. **Công bố cấu hình ECH qua DNS**: Máy chủ phát hành `ECHConfig` trong bản ghi **DNS** `HTTPS/SVCB` **(RFC 9848, RFC 9460)** gồm khóa công khai HPKE, thuật toán `KEM/KDF/AEAD`, `config_id` và `public_name`.
2. **Client tạo** `ClientHelloInner`: Thông điệp thật chứa SNI thật cần được bảo vệ.
3. **Client mã hóa** `ClientHelloInner`: Dùng khóa công khai HPKE nhận từ DNS để mã hóa.
4. **Client tạo** `ClientHelloOuter`: Đóng gói thông tin công khai kèm bản mã của `ClientHelloInner`.
5. **Máy chủ giải mã**: Dùng khóa riêng giải mã `ClientHelloInner`:
   - ✓ **Thành công**: Chấp nhận ECH, dùng `ClientHelloInner` tiếp tục Handshake và hoàn tất kết nối TLS.
   - ✗ **Lỗi / Không hỗ trợ**: Từ chối ECH; Client kiểm tra xác nhận và thử lại với cấu hình ECH mới nhất.

Nguy Cơ Lượng Tử Với TLS & Ba Hướng PQC

Giới Hạn PFS Trước Máy Tính Lượng Tử

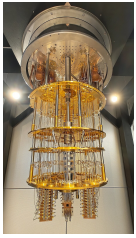
- **Vượt qua PFS (*Store now, decrypt later*)**: Dù (EC)DHE bảo vệ được khóa dài hạn, kẻ địch thu thập bản ghi hôm nay có thể chờ máy tính lượng tử bẻ khóa tạm ECDHE về sau.
- **Giả mạo xác thực**: Máy tính lượng tử đủ mạnh có thể giả chữ ký RSA/ECDSA và giả danh CA.

Thuật Toán Shor vs. Grover Trên TLS

- **Thuật toán Shor**: Phá phân tích số nguyên (RSA) và logarit rời rạc (DH/ECDHE/ECDSA).
- **Thuật toán Grover (Đối xứng & Băm)**: Ảnh hưởng nhẹ hơn: $\mathcal{O}(2^n) \rightarrow \mathcal{O}(2^{n/2}) \rightarrow$ tăng kích thước khóa (AES-256, SHA-384).
- **Trọng tâm**: Thay đổi phần **Handshake**; tầng **Record AEAD** chỉ cần tăng tham số.

Post-Quantum Cryptography (PQC): Mật mã dựa trên bài toán khó với cả máy tính cổ điển lẫn lượng tử; TLS cần ưu tiên thay thế thành phần **mật mã khóa công khai** ở quá trình bắt tay.

Phần Cứng Lượng Tử Thực Tế



IBM Quantum System One (Wikimedia, CC BY 4.0).

Hướng 1: PQ Authentication

Thay chữ ký cổ điển bằng chữ ký hậu lượng tử (ML-DSA/SLH-DSA) để xác thực.

Hướng 2: PQ Key Exchange

Thay trao đổi khóa truyền thống (DH/ECDH) bằng cơ chế **KEM** hậu lượng tử (ML-KEM).

Hướng 3: PQ-AKE

Kết hợp trao đổi khóa và xác thực hậu lượng tử trong cùng một thiết kế (KEMTLS).

PQ Authentication & PQ Key Exchange (KEM)

Hướng 1: Post-Quantum Authentication

- **2 vị trí dùng chữ ký số trong TLS**: (1) Chữ ký của CA trên chứng chỉ; (2) Chữ ký `CertificateVerify` trong Handshake.
- **Thách thức**: Cân bằng an toàn và hiệu năng, đặc biệt là kích thước khóa và chữ ký.

Thuật toán	PK (B)	Chữ ký (B)	Chậm tương đối*
Falcon (L1)	897	666	0,98×
Dilithium (L2)	1 312	2 420	1,02×
SPHINCS+ (L1)	32	7 856	1,58×

*Nguồn [14] (Phụ lục A, bảng 4–6): Hệ số = kết nối/giây(`Ed25519`) ÷ kết nối/giây(thuật toán ký); lớn hơn 1 là chậm hơn `Ed25519` (OQS-OpenSSL).

- **Nhận xét**: `Dilithium` (**ML-DSA** / **FIPS 204**) và `Falcon` cân bằng tốt nhất; `SPHINCS+` (**SLH-DSA** / **FIPS 205**) có chữ ký lớn.
- **Hybrid authentication**: Ghép chữ ký cổ điển + hậu lượng tử; mức chậm thay đổi tùy cặp thuật toán kết hợp [14].

Hướng 2: Trao Đổi Khóa Lai (RFC 10024)

- **Cơ chế nhóm lai**: Ghép **ML-KEM-768** (FIPS 203) và **X25519** (RFC 7748) để đạt **an toàn kép**: an toàn chừng nào $\geq$ 1 thuật toán chưa bị bẻ gãy.
- **Đóng gói hai chiều**: Kích thước chia tách rõ rệt giữa khóa công khai và bản mã [RFC 10024, mục 4]:



- **Khớp thực chứng**: Server share **1.120 B** kết nối trực tiếp với trường `key_share` trong gói tin Wireshark ở **Slide 28**.
- **Hiệu năng**: Hệ số chậm tương đối của Kyber L1 chỉ **1,06**× so với X25519 (xem **Phụ lục** cho benchmark đầy đủ).

Hướng 3: PQ-AKE (KEMTLS) & Vấn Đề Còn Mở

Bản Chất AKE & Rủi Ro TLS 1.3 Truyền Thống

Nhiệm vụ	Cơ chế cũ	Rủi ro trước lượng tử
Trao đổi khóa	`ECDHE`	Bị phá vỡ bởi thuật toán Shor
Xác thực	`RSA/ECDSA`	Có thể bị giả mạo nếu lượng tử đủ mạnh

- **Hai vấn đề bắt tay**: *Secrecy* (khóa phiên bí mật) & *Authenticity* (xác minh đúng Server).
- **Yêu cầu AKE**: Xác thực danh tính; tạo khóa bí mật chung; ngăn giả mạo/can thiệp; bảo vệ khóa phiên trên mạng không an toàn.

Giải Pháp PQ-AKE: Giao Thức KEMTLS

- **Ý tưởng cốt lõi**: Ký thông điệp là một cách xác thực, nhưng **không phải cách duy nhất**.
- **Cơ chế KEMTLS**: Thay vì dùng chữ ký số chứng minh danh tính Server, KEMTLS dùng chính **KEM** để chứng minh Server sở hữu khóa bí mật khớp với khóa công khai đã chứng thực trong certificate.



1. Phân Tích Hình Thức

Chưa có phân tích đầy đủ cho TLS với KEM làm trao đổi khóa (đặc biệt *hybrid*) và TLS với nhiều chữ ký (*hybrid authentication*).

2. Đa Dạng Hóa Giải Pháp

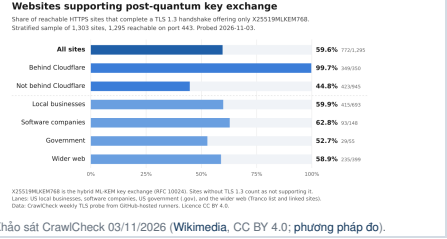
Cần bổ sung các giải pháp mật mã hậu lượng tử có chi phí truyền thông thấp cho những môi trường mạng kém.

3. Trạng Thái Chuẩn Hóa

Trao đổi khóa lai đã chuẩn hóa trong TLS 1.3: **RFC 10024** đặc tả các nhóm lai cụ thể (`X25519MLKEM768`), còn **RFC 9954** (trạng thái *Informational*) mô tả cấu trúc chung; **KEMTLS** là bản nháp IETF (`draft-celi-wiggers-tls-authkem`) [12].

Thực Chứng Triển Khai: TLS 1.3 & PQ-TLS (11/2026)

Tỷ Lệ Website Hỗ Trợ Trao Đổi Khóa Lai X25519MLKEM768

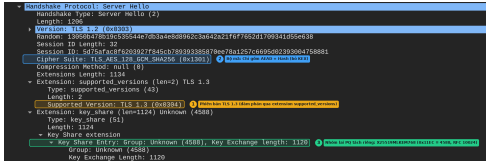


Khảo sát CrawlCheck 03/11/2026 (**Wikimedia**, CC BY 4.0; **phương pháp đo**).

Thực Trạng Chuyển Dịch PQ-TLS (RFC 10024)

59,6% (**772/1.295** website truy cập được trong mẫu 1.303 site; **99,7%** sau Cloudflare) hoàn tất bắt tay TLS 1.3 khi client **chỉ đề xuất** `X25519MLKEM768` (CrawlCheck [16]).

Bắt Tay Wireshark Thật: ServerHello Chọn X25519MLKEM768



Gói `ServerHello`: Nhóm trao đổi khóa nằm ở trường `key_share` riêng với bộ mã.

Phân Tích 3 Trường Trọng Tâm Trong ServerHello

- **1. Phiên bản TLS 1.3**: Đàm phán qua extension `supported_versions` (0x0304) [RFC 9846].
- **2. Bộ mã**: `TLS_AES_128_GCM_SHA256` chỉ chọn AEAD + băm; đã tách rời KEX.
- **3. Nhóm lai PQ**: `X25519MLKEM768` (`0x11EC`) dài 1.120 B trong `key_share` [RFC 10024].

Phụ Lục: Benchmark Hiệu Năng KEM Hậu Lượng Tử (NIST L1)

Hiệu Năng Các Cơ Chế KEM Mức An Toàn 1 (NIST Level 1)

Thuật toán KEM (Mức 1)	PK (B)	Ciphertext (B)	Chậm tương đối*
Kyber (L1) / ML-KEM-768	**800**	**768**	**1,06×**
Saber	672	736	1,07×
NTRU	930	930	1,13×
HQC	2 249	4 481	1,23×
FrodoKEM	9 616	9 720	2,53×
BIKE	1 540	1 572	2,55×

*Nguồn: N. Alnahawi et al. [14] (Phụ lục A, bảng 4–6): Hệ số chậm tương đối = kết nối/giây(`X25519`) ÷ kết nối/giây(KEM). Đo kiểm thực nghiệm trên môi trường OQS-OpenSSL.

Phân Tích Đánh Đổi An Toàn & Băng Thông

- **ML-KEM (Kyber)**: Cân bằng tối ưu giữa kích thước khóa/bản mã (800 B + 768 B) và tốc độ tính toán (1,06×), được NIST chuẩn hóa ưu tiên (**FIPS 203**).
- **Cấu trúc mạng tinh thể**: `Kyber`, `Saber`, `NTRU` dựa trên bài toán Module-LWE / NTRU, cho thông lượng cao vượt trội.
- **Mã hóa dựa trên mã sửa sai**: `HQC`, `BIKE` có bản mã lớn ($>$ 1,5 KB), làm tăng phân mảnh TCP.
- **Mạng phi cấu trúc**: `FrodoKEM` độ an toàn cao nhưng khóa rất lớn ($>$ 9 KB), độ trễ gấp 2,5×.

Danh Mục Tài Liệu Tham Khảo

[1] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF RFC 9846 (obsoletes RFC 8446), Jul. 2026.

[2] T. Dierks and E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.2,” IETF RFC 5246, Aug. 2008; J. Salowey *et al.*, RFC 5288.

[3] K. Moriarty and S. Farrell, “Deprecating TLS 1.0 and TLS 1.1,” IETF BCP 195 / RFC 8996, Mar. 2021; R. Barnes *et al.*, RFC 7568; RFC 6176.

[4] Y. Sheffer, P. Saint-Andre, and T. Fossati, “Recommendations for Secure Use of TLS and DTLS,” IETF RFC 9325, Nov. 2022; N. Aviram, RFC 10015, Jul. 2026.

[5] K. Kwiatkowski *et al.*, “Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3,” IETF RFC 10024, Aug. 2026; RFC 9954, Jul. 2026.

[6] E. Rescorla *et al.*, “The Datagram Transport Layer Security (DTLS) Protocol Version 1.3,” IETF RFC 9147, Apr. 2022; RFC 6347, Jan. 2012.

[7] E. Rescorla *et al.*, “TLS Encrypted Client Hello (ECH),” IETF RFC 9849, Mar. 2026; RFC 9848 (DNS ECH), Mar. 2026; RFC 9180; RFC 9460.

[8] D. Cooper *et al.*, “Internet X.509 PKI Certificate and CRL Profile,” IETF RFC 5280, May 2008; RFC 9525 (Service Identity); RFC 7627; RFC 8470.

[9] NIST, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS PUB 203 (with FIPS 204 ML-DSA, FIPS 205 SLH-DSA), Aug. 2024.

[10] NIST, “Guidelines for the Selection, Configuration, and Use of Transport Layer Security (TLS) Implementations,” NIST SP 800-52 Rev. 2, Aug. 2019.

[11] N. Modadugu and E. Rescorla, “The Design and Implementation of Datagram TLS,” in *Proc. 11th Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2004.

[12] P. Schwabe, D. Stebila, and T. Wiggers, “Post-Quantum TLS Without Handshake Signatures,” *ACM CCS*, 2020; IETF `draft-celi-wiggers-tls-authkem`.

[13] K. Bhargavan, V. Cheval, and C. Wood, “A Symbolic Analysis of Privacy for TLS 1.3 with Encrypted Client Hello,” in *Proc. ACM CCS*, 2022, pp. 365–379.

[14] N. Alnahawi *et al.*, “A Comprehensive Survey on Post-Quantum TLS,” *IACR Commun. Cryptol.*, vol. 1, no. 2, art. 6, 2024 (PDF).

[15] W. Stallings, *Network Security Essentials: Applications and Standards*, 6th ed. Pearson, 2017, ch. 5.

[16] Mozilla Firefox (MPL 2.0) & Wikimedia Commons; CrawlCheck PQ-TLS Probe 2026 (CC BY 4.0): crawlcheck.io/blog/...#how-this-was-measured.

CẢM ƠN CÔ VÀ CÁC BẠN ĐÃ LẮNG NGHE!

PHIÊN THẢO LUẬN & CÂU HỎI (Q&A)